Performance Analysis of Planar-Float and Direct Packed-BMP AVX2 Implementations for Image Grayscale Conversion

CEPARCO — Integrating Project
Abdulrahman, Lim, Lopez, Nicolas

Commented [1]: New title emphasizing planar-float vs packed-BMP

0. Abstract

Commented [2]: Comparison of three implementations, BMP handling, and performance methodology

Grayscale conversion applies the same weighted color calculation to each image pixel, making it suitable for data-level parallelism. Although direct AVX2 grayscale implementations already exist, their performance can depend strongly on how input pixels are represented in memory. This project proposes a controlled comparison of three implementations: a sequential C reference, a planar-float SIMD kernel that operates on separated color arrays, and a direct packed-BMP SIMD kernel that reads interleaved BGR bytes without first creating full planar-float arrays.

All three will use a common grayscale formula and explicitly defined byte-conversion rules. The program will support uncompressed 24-bit BMP images, including scanline padding, top-down and bottom-up orientation, and widths not divisible by eight. The study will measure both kernel execution and complete in-memory processing, including format-conversion overhead, over multiple image resolutions. Pixel-level comparisons will establish correctness, while timing and speedup measurements will reveal when each memory representation is beneficial.

Through these comparisons, the project aims to demonstrate the advantages and limitations of SIMD-based parallel computing in a practical image processing application.

1. Description of the Proposed Project

Commented [3]: Data representation problem and revised research question

Grayscale conversion is a common image-processing operation used in photography, document scanning, and computer vision. Its repeated per-pixel arithmetic makes it suitable for SIMD parallelization, but the organization of image data in memory can significantly affect processing performance.

Standard 24-bit BMP images store pixels as three interleaved blue, green, and red (BGR) bytes, with padding added at the end of each row when necessary. In contrast, SIMD floating-point operations can be performed more conveniently when the color channels are stored in separate arrays. Rearranging BMP pixel data into these arrays simplifies vector arithmetic but introduces additional preprocessing time and memory usage. This project proposes comparing two SIMD-based approaches to grayscale conversion: planar-float processing and direct packed-BMP processing. The goal is to determine whether the benefits of simplified SIMD arithmetic outweigh the preprocessing costs associated with separating image channels.

Research Question

How does processing data as planar floating-point channels compare with processing 24-bit packed BMP pixels directly, both in AVX2 kernel execution time and complete in-memory grayscale conversion?

Rather than assuming that one approach is always faster, the project will evaluate the advantages and disadvantages of each implementation using controlled performance measurements.

Objectives

1. Implement a correct sequential C implementation of grayscale conversion.
2. Develop two NASM SIMD kernels for planar-float and direct packed-BMP inputs.
3. Define consistent byte-level conversion and correctness verification criteria.
4. Support BMP boundary conditions, including row padding, image orientation, and widths that are not multiples of eight.
5. Analyze SIMD speedup, data representation, and preprocessing overhead across different image resolutions.

Commented [4]: Revised objectives involving two SIMD approaches and correctness

A. Inputs of the Project

Commented [5]: BMP specifications, row-stride formula, orientation, boundary tests

The primary inputs will be uncompressed 24-bit Windows BMP images (BI_RGB) containing three BGR bytes per pixel. Each image provides information such as its width, signed height, pixel-data offset, and row stride.

The program will validate the image headers, dimensions, supported color depth, compression type, and pixel-buffer boundaries before processing. Unsupported image formats will be rejected to prevent incorrect interpretation of pixel data. The proposed image resolutions include 640 × 480, 1280 × 720, 1920 × 1080, and 3840 × 2160 pixels. These resolutions will be used to evaluate how processing performance changes as the amount of image data increases.

BMP rows are padded to four-byte boundaries. For an image with width W, the row stride is calculated as:

$$RowStride = 4\left\lceil \frac{3W}{4} \right\rceil$$

This is equivalent to 4 * ((3W + 3) / 4) using integer division.

A positive BMP height indicates bottom-up storage, while a negative height indicates top-down storage. The program will interpret both formats correctly and preserve the logical pixel positions in the resulting grayscale image.

In addition to standard resolutions, smaller test images with widths of 1, 7, 8, 9, 13, and 17 pixels will be prepared. These will help verify row padding, memory boundaries, and the handling of remaining pixels that cannot form a complete eight-element SIMD vector. The test images will also include different color patterns, such as pure red, green, blue, black, white, and grayscale gradients, to verify correct channel interpretation and output conversion.

B. Proposed Process

Commented [6]: Rounding, clamping, and three implementation approaches

The weighted grayscale value is:

$$Y = 0.299R + 0.587G + 0.114B$$

The grayscale intensity will be calculated using the red, green, and blue components of each pixel, with input values ranging from 0 to 255. All three implementations will use single-precision floating-point arithmetic and follow the same calculation order as closely as possible to minimize differences in the resulting values.

After calculating the grayscale intensity, the result will be clamped to the range of 0 to 255 and rounded to the nearest integer using the ties-to-even rule. Truncation toward zero will not be used for the final grayscale value. The resulting 8-bit intensity will then be written into the blue, green, and red channels of the output pixel. These rules will apply consistently to the sequential C implementation, the planar-float AVX2 implementation, and the direct packed-BMP AVX2 implementation to support accurate output comparison.

The input BMP image will be read into memory once. The sequential C implementation will process its BGR values directly and generate grayscale output pixels. For the planar-float approach, the program will first separate the BGR

components into individual floating-point arrays before performing SIMD arithmetic and converting the results into output bytes. Meanwhile, the direct packed-BMP approach will extract the BGR components from the original interleaved pixel data and calculate grayscale intensities without creating full floating-point channel arrays. Both SIMD implementations will handle complete vector groups and process remaining pixels separately to ensure correctness for image widths that are not divisible by eight.

i. Processes to Be Parallelized Using SIMD

Commented [7]: Explanation of both AVX2 implementations

The primary process to be parallelized is the grayscale intensity calculation, which involves performing the same arithmetic operations independently on each image pixel. SIMD instructions allow these operations to be applied to multiple pixel values simultaneously, reducing the number of iterations needed to process the image.

For the planar-float implementation, the blue, green, and red components will first be separated into individual floating-point arrays. The SIMD kernel will then load groups of eight 32-bit floating-point values into 256-bit YMM registers and perform the required multiplication and addition operations to calculate grayscale intensities. For the direct packed-BMP implementation, the SIMD kernel will process the interleaved BGR pixel data without first creating complete floating-point channel arrays. This approach requires additional instructions to extract and rearrange the individual color components but avoids the preprocessing overhead associated with full planar conversion.

Both SIMD implementations will use equivalent grayscale calculations and output-conversion rules. Pixels that do not fit into complete SIMD groups will be handled separately to ensure correct processing regardless of image width. Through these implementations, the group aims to investigate how different pixel data representations influence the efficiency of SIMD-based grayscale conversion.

ii. Existing Implementations and Related Works

Commented [8]: New comparison of existing implementations and research contribution

Image grayscale conversion is a widely used technique in digital image processing, particularly in photography, document scanning, and computer vision. Since each pixel can be processed independently, grayscale conversion is suitable for Single Instruction, Multiple Data (SIMD) parallel computing, allowing multiple pixel values to be processed simultaneously.

Several existing image-processing libraries already provide optimized grayscale conversion methods. One example is OpenCV, an open-source computer vision library that supports various image-processing operations. Through its cvtColor() function, OpenCV can convert colored images into grayscale using a weighted calculation based on the red, green, and blue components of each pixel. Another relevant implementation is the Simd Library, which uses SIMD instruction sets such as SSE, AVX, and AVX2 to accelerate image-processing operations. The library includes optimized routines for color conversion and other pixel-based computations, demonstrating how parallel processing can improve performance when handling large amounts of image data.

Although grayscale conversion using SIMD is already established, the organization of image data can affect its performance. For instance, uncompressed 24-bit BMP images store pixel values in an interleaved Blue-Green-Red (BGR) format. One approach is to separate these components into individual floating-point arrays before processing them using AVX2 instructions. This planar-float approach simplifies vector arithmetic but requires additional memory and preprocessing. Alternatively, a direct packed-BMP approach processes the interleaved BGR values without first converting the entire image into separate floating-point arrays. While this reduces preprocessing overhead, it requires additional instructions to extract and rearrange pixel components for SIMD computation. These approaches introduce a trade-off between computational simplicity and memory-processing overhead. Although the planar-float approach may simplify SIMD operations, the direct packed-BMP approach may achieve better overall performance by avoiding unnecessary data preparation.

The proposed project will investigate these trade-offs by implementing three grayscale conversion methods: sequential C, planar-float AVX2, and direct packed-BMP AVX2. The SIMD implementations will be developed manually using NASM x86-64 assembly language to provide greater control over vector operations and data handling. Although optimized grayscale conversion implementations already exist, this project will focus on comparing the computational and memory-processing costs of two different input representations under consistent testing conditions. Both kernel execution time and complete in-memory processing time will be evaluated to determine whether the additional preprocessing required by planar-float conversion affects its overall efficiency. Therefore, the project does not claim to introduce a new grayscale conversion algorithm. Instead, its contribution will be a reproducible performance comparison that examines how data organization, preprocessing overhead, and SIMD execution influence grayscale conversion performance.

iii. Plans for Parallelizing the Proposed Process

Commented [9]: Revised implementation stages, correctness and performance protocols

The implementation will proceed through the following stages:

1. Implement a BMP reader and writer in C that validates image headers, BGR ordering, row padding, signed image height, pixel offsets, and buffer boundaries.
2. Develop the sequential C implementation and verify its grayscale outputs using manually calculated test pixels.
3. Implement the planar-float channel extraction process and develop a NASM grayscale conversion kernel using YMM registers.
4. Develop a separate direct packed-BMP NASM kernel that processes interleaved BGR pixels without allocating full planar-float channel arrays.
5. Implement boundary handling for image widths that are not multiples of eight and verify correct top-down and bottom-up processing.
6. Compare the outputs of all implementations and conduct performance measurements across different image resolutions.

Correctness Protocol

The primary correctness requirement is exact byte-for-byte equality between the output pixels of the sequential C implementation and both SIMD implementations. For each test image, the program will compare the valid grayscale pixel bytes and record the total number of mismatches, mismatch percentage, and maximum absolute byte difference.

If minor differences arise because of floating-point operation order or rounding behavior, differences of up to one grayscale intensity level will be recorded separately for analysis. However, these outputs will not be classified as exact matches. Row-padding bytes will be excluded from pixel comparison because they do not represent image information.

Performance Protocol

The group will evaluate performance using two measurement categories: kernel-only execution time and complete in-memory processing time.

Kernel-only measurements will focus on the main grayscale computation performed by each implementation using its required input representation. However, since the planar-float and direct packed-BMP kernels operate on different data layouts, these measurements will not be treated as fully equivalent measures of overall processing efficiency.

Complete in-memory measurements will include all necessary processing stages, beginning with the original packed BMP pixel data and ending with the final grayscale pixel buffer. This includes planar channel extraction, numerical conversion, and output-byte preparation where applicable. Disk file reading and writing will be excluded to avoid introducing storage-related timing variations.

All implementations will be tested using the same input images and an optimized x64 Release build. Each configuration will undergo at least 30 measured executions after warm-up. The group will calculate mean and median execution times and determine speedup relative to the sequential C reference using equivalent measurement boundaries.

Compiler optimization settings and possible automatic vectorization of the C reference will also be documented to ensure that performance results are interpreted correctly.

The speedup will be calculated using:

Speedup = Sequential C Execution Time / SIMD Execution Time

Separate speedup values will be reported for kernel-only and complete in-memory measurements to avoid comparing different amounts of processing work.

iv. Processes That Will Not Be Parallelized

Commented [10]: Preprocessing costs included in performance evaluation

BMP file access, header validation, memory allocation, test orchestration, and result reporting will remain in C. The planar-float conversion can initially run in sequential C, but its execution time will always be included in the complete in-memory planar measurement.

Direct packed-BMP row traversal and remainder handling may use scalar C or NASM instructions where necessary. Common output formatting and correctness checking will be applied consistently across implementations and documented in the evaluation. Although certain preprocessing operations will remain sequential, their execution times will be included in the complete in-memory performance comparison to ensure that the evaluation reflects the full processing cost of each implementation.

C. Outputs of the Project

Commented [11]: Three implementations, correctness metrics, and benchmarking outputs

The expected project outputs include:

1. Grayscale BMP images for each supported input, preserving correct image orientation.
2. Three working implementations: sequential C, planar-float AVX2, and direct packed-BMP AVX2.
3. Correctness tables showing exact-byte match status, mismatch count, mismatch percentage, and maximum byte difference.
4. Separate kernel-only and complete in-memory execution-time tables for different image resolutions, including average times, median times, and calculated speedup values.
5. Performance graphs and an analysis of conversion overhead, memory access, SIMD execution, and boundary handling.

These outputs will be used to determine how pixel data representation affects SIMD processing performance and to identify the conditions under which each implementation provides the greatest benefit.

2. Proposed Target Implementation Platform

Commented [12]: Updated SIMD and data-layout specifications

Component	Proposed Platform
Programming language	C for reference implementation, BMP I/O, and benchmarking; x86-64 NASM assembly for SIMD kernels
Parallel implementation	CPU SIMD using 256-bit YMM registers and AVX/AVX2 instructions
Assembler	NASM
Development environment	Microsoft Visual Studio
Operating System	64-bit Windows
Build Configuration	Optimized x64 release
Processor	x86-64 CPU supporting AVX2
Input/output	24-bit uncompressed Windows BMP (BI_RGB)
Data Layout	Planar single-precision channels and direct packed BGR scanlines
Version control	GitHub

The main C program will handle image input/output, memory management, reference calculations, correctness verification, and performance measurement. The SIMD kernels will be developed as external NASM assembly modules and integrated into the Visual Studio project.

The final benchmarking environment will be documented, including the processor model, compiler configuration, and relevant hardware characteristics, to support reproducibility.

3. Mid-Project Milestones

Milestone 1 — Week 6: Project Proposal

The group will submit the revised project proposal outlining the two SIMD data representations, BMP image format requirements, numerical conversion policies, benchmarking methodology, and related work.

Milestone 2 — Week 10: Mid-Project Update and Demo

The group aims to demonstrate a functional BMP reader/writer and sequential C grayscale conversion program. The implementation will be tested using images with different widths, row-padding requirements, and positive or negative BMP heights to verify correct channel ordering and image orientation.

The group also aims to complete the initial planar-float AVX2 implementation and present preliminary progress on direct packed-BMP processing, including a working prototype or successful extraction of BGR values using SIMD instructions.

Milestone 3 — Week 13: Complete Project Submission and Demo

The group aims to complete and verify both SIMD kernels, integrate them into the main C program, and conduct correctness and performance experiments across different image resolutions.

The group will analyze kernel-only and complete in-memory execution times, calculate speedup, and discuss the effects of memory representation and preprocessing overhead. The final project files, including the Visual Studio/NASM source code, test cases, benchmarking results, performance tables, and documentation, will be uploaded to GitHub.

4. References

[1] OpenCV, "Color Conversions," *OpenCV Documentation*. https://docs.opencv.org/4.13.0/de/d25/imgproc_color_conversions.html

[2] Simd Library, "Simd Image Processing Library," GitHub. https://github.com/ermig1979/Simd

[3] Intel Corporation, "Intel 64 and IA-32 Architectures Software Developer's Manuals." https://www.intel.com/content/www/us/en/developer/articles/technical/intel-sdm.html

[4] Microsoft Learn, "BITMAPINFOHEADER structure," *Microsoft Windows Documentation*. https://learn.microsoft.com/en-us/windows/win32/api/wingdi/ns-wingdi-bitmapinfoheader

Commented [13]: Revised Week 10 and Week 13 deliverables

Commented [14]: Newly added references and supporting citations